
Installing the System Scanner Client on macOS & Linux

Version 1.7.0 · Generated August 25, 2026 · Applies to Intel and Apple-silicon Macs and mainstream Linux distributions (Ubuntu, Debian, Fedora, Arch).

The client is a single self-contained native binary — no Python installation is required on the target machine. Each operating system gets its own build:

Windows	client_scanner.exe — served automatically when you open the download link on Windows.
----------------	--

Linux	system-scanner_1.7.0_amd64.deb — native Debian/Ubuntu package, installs with one command. Other distros can grab client_scanner-linux.zip .
--------------	---

macOS	client_scanner-macos.zip — contains a proper System Scanner.app bundle.
--------------	---

How downloads work: Open your admin panel and click **Download Client**. The panel detects your operating system automatically and serves the right file. You can force a specific build with `/download-client/?os=linux` or `/download-client/?os=macos`.

Part A — macOS Installation (step by step)

Step 1: Download. In your browser, go to your admin panel URL and click **Download Client**. Safari/Chrome on a Mac receives **client_scanner-macos.zip** automatically.

Step 2: Unzip it. Double-click the ZIP in Finder — a **System Scanner.app** appears — or run:

```
cd ~/Downloads
unzip client_scanner-macos.zip
cd system-scanner-macos
```

Step 3: Allow the app to run (Gatekeeper). Because the app is not notarised with an Apple Developer certificate, macOS may block first launch. Remove the quarantine flag:

```
xattr -dr com.apple.quarantine "System Scanner.app"
```

If System Settings blocked it anyway: Go to **System Settings** → **Privacy & Security**, scroll to the Security section and click **Allow Anyway** next to the blocked app message, then relaunch.

Step 4: Start the client (first run from Terminal). The console prints your Registration Key, so start it once via Terminal to see it. Double-clicking the .app also works (it runs in the background; output goes to the system log):

```
"System Scanner.app/Contents/MacOS/client_scanner-macos"
# after approval you can simply:
open "System Scanner.app"
```

Step 5: Register and wait for approval. The console prints a Registration Key (e.g. **A1B2C3D4**) and shows “Waiting for admin approval...”. Open the admin dashboard, find your Mac under **Pending**, and click **Approve**. The client turns green (Online) within one scan interval.

Optional — start automatically at login (macOS)

Create a LaunchAgent so the scanner starts whenever you log in:

```
mkdir -p ~/Library/LaunchAgents
cat > ~/Library/LaunchAgents/com.systemscanner.client.plist << 'EOF'
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE plist PUBLIC "-//Apple//DTD PLIST 1.0//EN"
    "http://www.apple.com/DTDs/PropertyList-1.0.dtd">
<plist version="1.0"><dict>
  <key>Label</key><string>com.systemscanner.client</string>
  <key>ProgramArguments</key>
  <array><string>/Users/YOU/system-scanner-macos/System Scanner.app/Contents/MacOS/client_scan
  <key>RunAtLoad</key><true/>
  <key>KeepAlive</key><true/>
</dict></plist>
EOF
launchctl load ~/Library/LaunchAgents/com.systemscanner.client.plist
```

Part B — Linux Installation (step by step)

Step 1 **Download.** From the admin panel click **Download Client** — Linux browsers get the **system-scanner_1.7.0_amd64.deb** package (Debian/Ubuntu). Fetch it directly with wget:

```
# replace YOUR-PANEL with your admin server URL
wget 'https://YOUR-PANEL/download-client/?os=linux' -O system-scanner_1.7.0_amd64.deb
```

Step 2 **Install the package.** This installs the binary to **/usr/local/bin/system-scanner**:

```
sudo dpkg -i ./system-scanner_1.7.0_amd64.deb
# if a dependency error appears:
sudo apt -f install
```

Not on Debian/Ubuntu? Fedora, Arch, etc. can download the portable zip instead: <https://YOUR-PANEL/download-client/?os=linux&format=zip>, then unzip and run `chmod +x client_scanner-linux && ./client_scanner-linux`.

Step 3 **Run it:**

```
sudo system-scanner
```

Step 4 **Get approved.** Note the Registration Key printed in the terminal. On the admin dashboard your machine appears with a yellow **Pending** dot — click **Approve**. Heartbeats and scans begin immediately after approval.

Step 5 **Verify connectivity.** If nothing appears on the dashboard, confirm the panel is reachable from the machine:

```
curl -I https://YOUR-PANEL/api/health # expect HTTP 200
```

Optional — run as a background service (Linux)

Install as a systemd service so it survives reboots and runs headless:

```
sudo tee /etc/systemd/system/system-scanner.service > /dev/null << 'EOF'
[Unit]
Description=System Scanner Pro Client
After=network-online.target
Wants=network-online.target

[Service]
ExecStart=/usr/local/bin/system-scanner
Restart=always
RestartSec=10
User=root

[Install]
WantedBy=multi-user.target
EOF
sudo systemctl daemon-reload
sudo systemctl enable --now system-scanner
journalctl -u system-scanner -f
```

Troubleshooting

Symptom	Cause & fix
"Permission denied" on Linux/macOS	Binary not executable — run <code>chmod +x client_scanner-*</code> .
"App can't be opened" on macOS	Gatekeeper quarantine — run <code>xattr -dr com.apple.quarantine "System Scanner.app"</code> , or use System Settings → Privacy & Security → Allow Anyway.
dpkg reports missing dependencies	Run <code>sudo apt -f install</code> to fix up, then retry. Or use the portable zip: <code>/download-client/?os=linux&format=zip</code> .
Client stuck at "Pending"	An admin must approve it from the dashboard, or the device re-registers each poll. Approve once — approval persists by registration key.
Client never appears on dashboard	Wrong panel URL configured, or outbound HTTPS blocked. Check <code>curl -I https://YOUR-PANEL/api/health</code> returns 200.
"cannot execute binary file"	You downloaded the build for a different OS — re-download from the panel so it matches yours.

Good to know: All builds talk to the same admin panel over HTTPS with automatic WebSocket→HTTP fallback, so no firewall rules beyond outbound 443 are needed. Deleting a device from the dashboard only soft-hides it; re-registration with the same key restores history.